

实验题目：音乐合成

班级：无 32

学号：2023010504

姓名：张乐

日期：2025 年 7 月 16 日

一、实验目的

1. 基于傅里叶级数和傅里叶变换等基础知识，应用第一篇讲授的 MATLAB 编程技术，在电子音乐合成方面做一些练习。
2. 增进对傅里叶级数的理解，并能够熟练运用 MATLAB 基本指令。
3. 对乐理和电子音乐的基本知识形成认知，并能将部分性质应用于编程分析中。

二、简单的音乐合成

1. 用一系列乐音拼接《东方红》片段

(1) 设计思路

《东方红》乐曲前四小节涉及到的音及对应频率如下：

$$1 = F \frac{2}{4} \quad | \quad 5 \quad \underline{56} \quad | \quad 2 \quad - \quad | \quad 1 \quad \underline{16} \quad | \quad 2 \quad - \quad |$$

bE	E	622.25 Hz	659.25 Hz	
bD	D	554.36 Hz	587.33 Hz	6
	C		523.25 Hz	5
bB	B	466.16 Hz	493.88 Hz	
bA	A	415.30 Hz	440 Hz (f_{A1})	
bG	G	369.99 Hz	392 Hz	2
	F		349.23 Hz	1
bE	E	311.13 Hz	329.63 Hz	
bD	D	277.18 Hz	293.66 Hz	6
	C		261.63 Hz (中央C, f_{C1})	.
bB	B	233.08 Hz	246.94 Hz	
bA	A	207.65 Hz	220 Hz (f_{A0})	
bG	G	184.99 Hz	196 Hz	
	F		174.61 Hz	

对于这段乐曲，指导书上写明“一拍约为 0.5s”。根据旋律中每个音的节拍和每一拍的时长，可以计算出每个音的持续时间。用幅度为 1、抽样频率为 8kHz 的正弦信号表示这些乐音，并用这一系列乐音信号拼出整段 rhythm 旋律，最终用 sound 函数播放《东方红》前四小节的旋律，并保存在 1_1.wav 文件中。

(2) 核心代码

```
fs = 8000;
beattime = 0.5; % 一拍约为 0.5s

freq = [523.25, 523.25, 587.33, 392, 349.23, 349.23, 293.66, 392]; % 对应旋律中每个音的频率
beatnum = [1, 0.5, 0.5, 2, 1, 0.5, 0.5, 2]; % 对应旋律中每个音的节拍
rhythm = [];

for i = 1 : length(beatnum)
    t = beatnum(i) * beattime; % 音符持续时间
    t = linspace(0, t, t * fs); % 采样点
    rhythm = [rhythm, sin(2 * pi * freq(i) * t)];
end

sound(rhythm, fs);
audiowrite('1_1.wav', rhythm, fs);
```

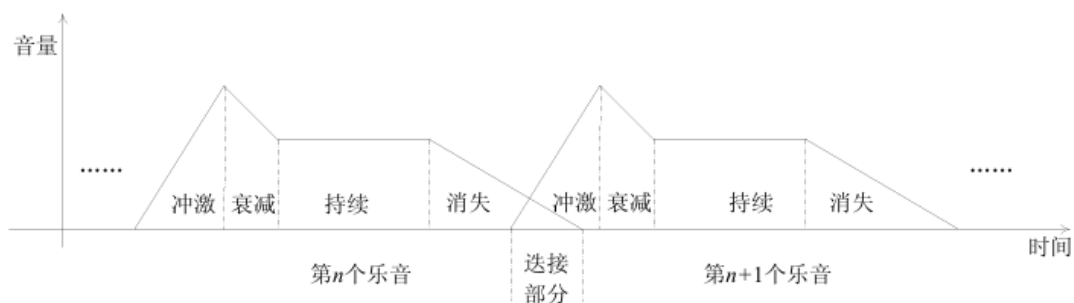
(3) 运行结果

生成的音调准确，旋律基本准确，但乐曲中相邻乐音之间有“啪”的爆破杂声。

2. 用包络修正乐音

(1) 设计思路

根据指导书中的包络示意图，设计了 3 种包络，分别为：原始音量变化包络，指数函数衰减形式包络，以及每一段是指数函数形式的音量变化包络，以满足实验要求。将 3 种包络修正后的旋律分别保存在 1_2_1.wav，1_2_2.wav，1_2_3.wav 文件中。



(2) 核心代码

```
fs = 8000;
beattime = 0.5;
```

```
freq = [523.25, 523.25, 587.33, 392, 349.23, 349.23, 293.66, 392];
beatnum = [1, 0.5, 0.5, 2, 1, 0.5, 0.5, 2];
rhythm = [];

for i = 1 : length(beatnum)
    t = beatnum(i) * beattime; % 音符持续时间
    t = linspace(0, t, t * fs); % 采样点
    rhythm = [rhythm, sin(2 * pi * freq(i) * t) .* envel(t,3)]; % 乘包络
end

sound(rhythm, fs)
audiowrite('1_2.wav', rhythm, fs);
% plot(t,sin(2 * pi * freq(i) * t) .* envel(t,3)); % 若想绘制包络作用后的
% 音频图像，则运行该行代码

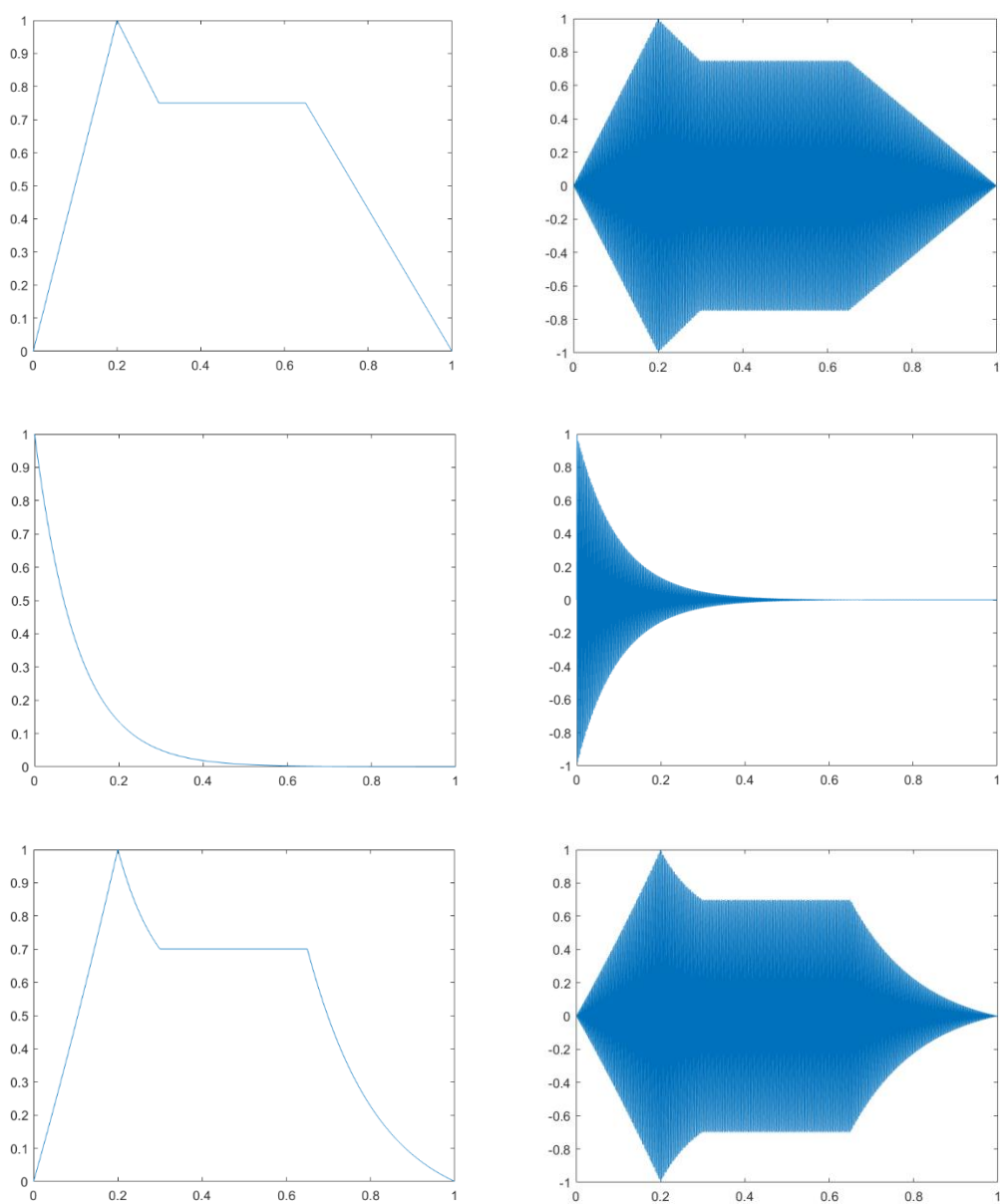
%% 包络线函数定义
function env = envel(t,n)
    switch n
    case 1 % 分段折线式的原始 ADSR 包络
        env = zeros(1, length(t));
        env(1:0.2*end) = t(1:0.2*end); % Attack: 20%时长
        env(1:0.2*end) = env(1:0.2*end)/env(0.2*end); % 归一化
        env(0.2*end+1:0.3*end) = -0.5*t(0.2*end+1:0.3*end) + 0.3; %
        Decay: 10%时长
        env(0.2*end+1:0.3*end) = env(0.2*end+1:0.3*end)/env(0.2*end +
        1); % 归一化
        env(0.3*end+1:0.65*end) = env(0.3*end); %Sustain: 35%时长
        env(0.65*end+1:end) = t(0.65*end+1:end); % Release: 35%时长
        env(0.65*end+1:end) = (env(0.65*end+1:end) -
        env(end))/(env(0.65*end+1)-env(end))* env(0.3*end); % 归一化
    case 2 % 指数函数
        env = zeros(1, length(t));
        env(1:end) = exp(-10*t(1:end));
    case 3 % 每一段是指数函数形式的 ADSR 包络
        env = zeros(1, length(t));
        env(1:0.2*end) = 1./exp(-t(1:0.2*end)) - 1; % Attack: 20%时长
        env(1:0.2*end) = env(1:0.2*end)/env(0.2*end); % 归一化
        env(0.2*end+1:0.3*end) = exp(-10*t(0.2*end+1:0.3*end)) + 0.15; %
        Decay: 10%时长
        env(0.2*end+1:0.3*end) = env(0.2*end+1:0.3*end)/env(0.2*end +
        1); % 归一化
        env(0.3*end+1:0.65*end) = env(0.3*end); % Sustain: 35%时长
        env(0.65*end+1:end) = exp(-6*t(0.65*end+1:end)); % Release: 35%
        时长
```

```
env(0.65*end+1:end) = (env(0.65*end+1:end)-  
env(end))/(env(0.65*end+1)-env(end))* env(0.3*end); % 归一化  
end  
% plot(t,env); % 若想绘制包络图像，则运行该行代码  
end
```

(3) 运行结果

乐音与乐音之间的爆破声消除，出现连续质感，效果较好。在 3 种包络中，每一段是指数函数形式的音量变化包络的效果最好，因此后续都使用该种包络。

每一种包络的图像与旋律经过包络后的图像如下图所示：



3. 八度变化处理

(1) 设计思路

将音乐升高和降低八度的简单方法是改变播放时的采样率频率 f_s ：若将采样率变为原来的 2 倍，生成的旋律会听上去升高了八度；若将采样率变为原来的 $1/2$ 倍，生成的旋律会听上去降低了八度。将升高八度的音乐保存在 1_3_1.wav 中，将降低八度的音乐保持在 1_3_2.wav 中。

若想用稍复杂的方法将音乐升高半个音阶，可使用 `resample` 函数。升高半音，则频率需变为原来的 $2^{(1/12)}$ 倍。用 `resample` 函数对信号按比例 p/q 重新采样，拉伸时间轴，使波形周期缩短，频率升高半音，播放时采样率保持不变。将升高半个音阶的音乐保存在 1_3_3.wav 文件中。

(2) 核心代码

```
%% 用简单方法将旋律进行八度变化
sound(rhythm, 2*fs); % 频率变为 2 倍，升高八度
audiowrite('1_3_1.wav', rhythm, 2*fs);
sound(rhythm, fs/2); % 频率变为 1/2 倍，降低八度
audiowrite('1_3_2.wav', rhythm, fs/2);
```

```
%% 用 resample 函数将旋律升高半个音阶
[p, q] = rat(2^(1/12)); % 找到最接近的有理分数近似 [p, q]
rhythm1 = resample(rhythm, p, q);
sound(rhythm1, fs);
audiowrite('1_3_3.wav', rhythm1, fs);
```

(3) 运行结果

可以听出音调的音高按照预期发生了变化。用简单方法升高（或降低）八度时，在音高改变的同时，音乐的速度会变快（或变慢）。理论上，用重采样方法改变音高也会改变音乐播放速率，但由于音高变化较小，速度变化并不太明显。

4. 为音乐增加谐波分量

(1) 设计思路

选择基波幅度为 1，二次谐波幅度为 0.2，三次谐波幅度为 0.3，在原频率上增加谐波频率。只需对第 2 小问的代码进行补充。将增加的谐波分量的音乐保存在 1_4.wav 中。

(2) 核心代码

```
for i = 1 : length(beatnum)
    t = beatnum(i) * beattime; % 音符持续时间
    t = linspace(0, t, t * fs); % 采样点
    env = envel(t,3);
    temp = sin(2 * pi * freq(i) * t) + 0.2 * sin(4 * pi * freq(i) * t) +
0.3 * sin(6 * pi * freq(i) * t);
    temp = temp .* env;
    rhythm = [rhythm, temp];
end
rhythm = rhythm / max(abs(rhythm)); % 归一化防止削波

sound(rhythm, fs)
audiowrite('1_4.wav', rhythm, fs);
```

(3) 运行结果

旋律的厚重感增加，且该谐波比例合成的声音与风琴较相似。

5. 自选音乐合成

(1) 设计思路

选择合成的曲目为《北京的金山上》。仿照第 2 小问的代码，修改乐音和节拍，包络不变，即可生成新曲目的音乐。将《北京的金山上》第一段的音乐保存在 1_5.wav 中。

北京的金山上

1=C $\frac{4}{4}$ 藏族民歌 马倬编词
优美地赞颂

(6[˙] 2̣[˙] 3̣[˙] 6[˙] 2̣[˙] 3̣[˙] | 3⁵ 6[˙] 2̣[˙] 2̣[˙] 1̣[˙] 6[˙] | 6 6⁶ 6 6⁶ |

6 6 [˙] 3̣[˙] 3̣[˙] 2̣[˙] | [˙] 2̣[˙] 3̣[˙] 2̣[˙] 2̣[˙] 1̣[˙] 6[˙] | 6 - - [˙] 1̣[˙] |

1. 北京的金山上光芒照四方，
2. 北京的金山上光芒照四方，

[˙] 3̣[˙] 2̣[˙] [˙] 6[˙] [˙] 1̣[˙] 2̣[˙] 3̣[˙] | 6 [˙] 1̣[˙] 2̣[˙] 6 [˙] 6⁵ 5³ | 3 - - |

毛主席就是金色的太阳。
毛主席就是金色的太阳。

6 6 5 3 6 6 5 3 | 6 6 6 [˙] 3̣[˙] 3̣[˙] 2̣[˙] 2̣[˙] 1̣[˙] 6[˙] |

多么温暖，多么慈祥，把翻身农奴的心儿照
多么温暖，多么慈祥，把翻身农奴的心儿照

6 - - | [˙] 1̣[˙] 6[˙] 2̣[˙] 3̣[˙] 6 6 5 3 | 3 5 6 [˙] 1̣[˙] 2̣[˙] 2̣[˙] 2̣[˙] 1̣[˙] 6[˙] |

亮。我们迈步走在社会主义幸福的大道
亮。我们迈步走在社会主义幸福的大道

1. 2. 6 - - :|| 6 - - 2̣[˙] 1̣[˙] 6[˙] | $\frac{2}{4}$ 6 6 6 0 ||

上。上。哎 巴扎 咳

(2) 核心代码

```
fs = 8000;
beattime = 0.5; % 一拍约为 0.5s

freq = [440, 440, 523.25, 659.25, 659.25, 587.33, 523.25, 587.33,
        659.25, 587.33, 587.33, 523.25, 523.25, 440, 440,...
        523.25, 659.25, 587.33, 523.25, 440, 523.25, 587.33, 659.25,
        440, 523.25, 587.33, 440, 440, 392, 392, 329.63, 329.63,...
        440, 440, 392, 329.63, 440, 440, 392, 329.63, 440, 440, 440,
        523.25, 659.25, 659.25, 587.33, 587.33, 587.33, 523.25, 523.25, 440,
        440,...
        523.25, 440, 523.25, 587.33, 659.25, 440, 440, 392, 329.63,
        329.63, 392, 440, 523.25, 587.33, 587.33, 587.33, 587.33, 523.25,
        523.25, 440, 440,...
        587.33, 523.25, 523.25, 440, 440, 440, 440]; % 对应旋律中每个音的
频率
beatnum = [1, 0.5, 0.5, 1, 0.5, 0.5, 1, 0.5, 0.5, 1, 0.25, 0.25, 0.25,
           0.25, 3,...
           1, 1, 0.5, 0.25, 0.25, 1, 0.5, 0.5, 1, 0.5, 0.5, 1, 0.25,
           0.25, 0.25, 0.25, 4,...
           0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.25, 0.25, 0.5,
           0.25, 0.25, 0.5, 0.5, 0.25, 0.25, 0.25, 0.25, 4,...
           0.5, 0.25, 0.25, 0.5, 0.5, 0.5, 0.25, 0.25, 1, 0.5, 0.5, 0.5,
           0.5, 0.5, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 4,...
           0.25, 0.25, 0.25, 0.25, 0.5, 0.5, 0.5]; % 对应旋律中每个音的节
拍
rhythm = [];

for i = 1 : length(beatnum)
    t = beatnum(i) * beattime; % 音符持续时间
    t = linspace(0, t, t * fs); % 采样点
    rhythm = [rhythm, sin(2 * pi * freq(i) * t) .* envel(t,3)]; % 乘包络
end

sound(rhythm, fs)
audiowrite('1_5.wav', rhythm, fs);
% plot(t,sin(2 * pi * freq(i) * t) .* envel(t,3)); % 若想绘制包络作用后的
音频图像, 则运行该行代码
```

(3) 运行结果

成功合成出了《北京的金山上》的旋律。

三、 用傅里叶级数分析音乐

1. 载入光盘中的 fmt.wav 文件

(1) 设计思路

用 audioread 函数载入 fmt.wav 文件，用 sound 函数播放这段音乐，并用 audiowrite 函数写入 1_6.wav 中。指导书上建议使用的 wavread 函数已变成了 audioread 函数。

(2) 核心代码

```
[rhythm, fs] = audioread('fmt.wav');  
sound(rhythm,fs);  
audiowrite('1_6.wav', rhythm, fs);
```

(3) 运行结果

播放的音乐效果很贴合实际吉他音色，相比原本合成的音乐更真实。

2. 从真实值 realwave 中通过预处理得到 wave2proc

(1) 设计思路

先从 Guitar.MAT 中载入 wave2proc 的理论值，并绘制 realwave 和 wave2proc 的图像。同时，自己设计代码对 realwave 进行预处理，包括以下几步：将采样率增加 100 倍进行 resample 重采样；将信号分成 10 段，每段通过取平均抑制噪声和非线性失真；减去直流分量；恢复原始信号长度和采样率。

(2) 核心代码

```
%% 载入样本  
load('Guitar.MAT');  
  
figure;  
plot([1:length(realwave)]/fs, realwave);  
xlabel("t(8kHz Sampled)");  
ylabel("amplitude");  
title("realwave");  
  
figure;  
plot([1:length(wave2proc)]/fs, wave2proc);
```

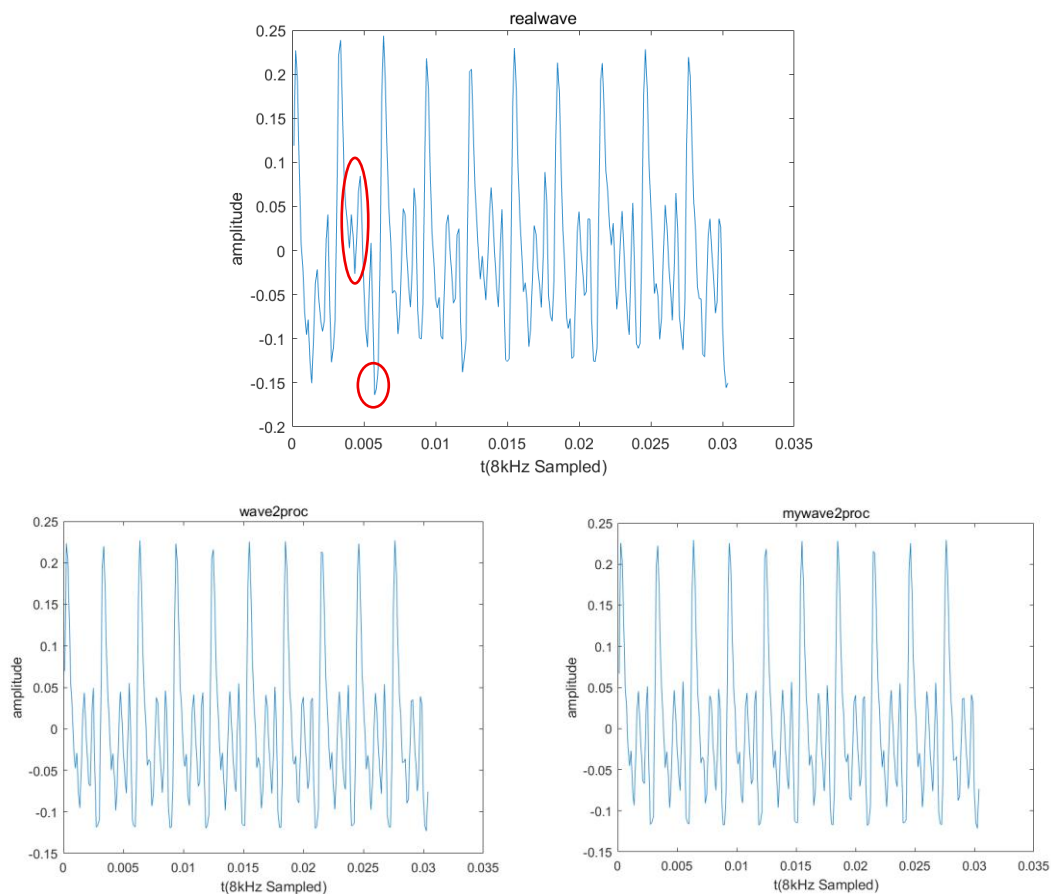
```
xlabel("t(8kHz Sampled)");
ylabel("amplitude");
title("wave2proc");

%% 预处理
fs = 8000;

wave = resample(realwave, fs*100, fs); % 重采样, 将采样率增加 100 倍
wave = reshape(wave, length(realwave)*10, 10); % 将信号分成 10 段
averwave = mean(wave, 2); % 抑制噪声和非线性失真
mywave2proc = averwave - mean(averwave); % 去除直流分量
mywave2proc = repmat(mywave2proc, 10, 1); % 恢复原始信号长度
mywave2proc = resample(mywave2proc, fs, fs*100); % 恢复原始采样率

figure;
plot([1:length(mywave2proc)]/fs, mywave2proc);
xlabel("t(8kHz Sampled)");
ylabel("amplitude");
title("mywave2proc");
```

(3) 运行结果



相较于真实信号 `realwave`，经过预处理后的确成功抑制了直流分量，并消除了部分非线性噪声。此外，自行设计代码进行预处理后生成的 `mywave2proc` 与给定的理论信号 `wave2proc` 几乎一致，说明设计的预处理方案有效。

3. 分析音乐的基频、音调和谐波分量

(1) 设计思路

对经过预处理的 `wave2proc` 信号做傅里叶变换 `fft`，由对称性、取绝对值等操作简化计算，通过 `findpeaks` 函数找到幅频特性曲线的峰值，找到每个峰值对应的频率位置，求出基波和谐波的频率。

(2) 核心代码

```
wave2proc_r = repmat(wave2proc,100,1); % 将时域信号重复 100 次

%% 进行傅里叶变换
fs = 8000;
L = length(wave2proc_r);
t = (0:L-1)/fs;
N = 2^nextpow2(L); % 寻找最接近的 2 的幂次，因为 FFT 在 N=2^n 时计算速度最快，
优化计算效率
wave_f = fft(wave2proc_r,N)/L; % 执行 FFT 并归一化
f = fs/2* linspace(0,1,N/2+1); % 频率：0~fs/2
amp = 2*abs(wave_f(1:N/2+1)); % 幅度：单边

plot(f,amp) % 绘制幅频图像
xlabel("frequency/Hz");
ylabel("amplitude");

%% 求基频和谐波分量
[pk,loc] = findpeaks(amp,'minpeakheight',0.002,'minpeakdistance',50); %
寻找峰值
freq_p = f(loc)
```

(3) 运行结果

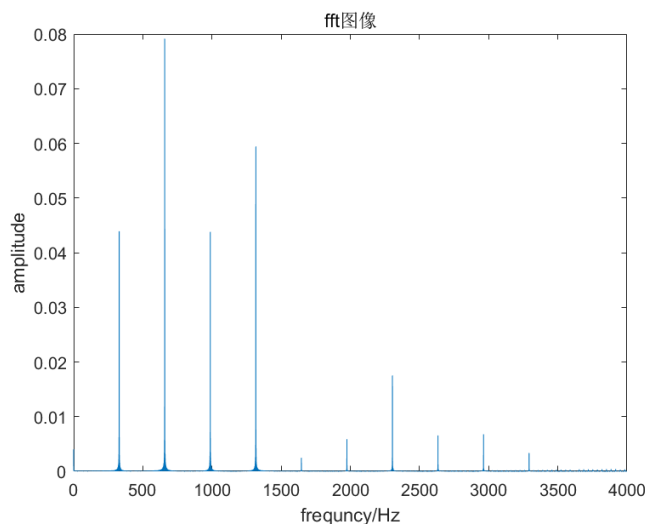
由运行结果可知，基波频率为 329.1Hz，对应音调为 E4，谐波频率包括：658.4Hz、987.5Hz、1316.9Hz、1646.0Hz、1975.3Hz、2304.4Hz、2633.8Hz、2962.9Hz、3292.2Hz。

```
>> HW1_8
```

```
freq_p =
```

```
1.0e+03 *
```

```
0.3291 0.6584 0.9875 1.3169 1.6460 1.9753 2.3044 2.6338 2.9629 3.2922
```



4. 自动分析 fmt.wav 文件的音调和节拍

(1) 设计思路

程序通过动态包络检测方法划分节拍，使用 `findpeaks` 函数定位包络的显著上升点，即对应音符起始时刻。

对每个音符段重复扩展后加 Hann 窗，通过 FFT 频谱分析提取峰值频率。基于容忍度 `tolerance` 排除谐波干扰，确定基频，并转换为对应音名。对于不合法的频率，会将音调显示为 Rest。

运行程序会绘制旋律的包络图像，并将判断出的音调结果、音的持续时间等信息输出在控制台，也会将信息存储在 `ans.csv` 和 `ans.mat` 文件中。此外，将所有功能用 `function result` 进行封装，为简化后面封装所有功能时的操作。

(2) 核心代码

```
function result = HW1_9()
    [y, fs] = audioread('fmt.wav');

    % 参数设置
    t_min = 0.08; % 最小音符持续时间
    f_min = 60; % 最低检测频率
```

```
f_max = 1200; % 最高检测频率
threshold = 0.015; % 音符起始检测阈值
tolerance = 0.05; % 谐波频率匹配容忍度

t = (0:length(y)-1)/fs;

%% 音符起始点检测
% 计算信号包络
w2 = abs(y);
w3 = conv(w2, hanning(round(fs/10)))/sum(hanning(round(fs/10)));
w4 = max(w3/max(w3), 0);

% 检测音符起始点
dynamic_threshold = max(threshold, 0.1*max(w4)); % 取固定阈值和动态阈值的较大者
[~, locs] = findpeaks(w4, 'MinPeakDistance', round(0.1*fs), ...
    'MinPeakHeight', dynamic_threshold, ...
    'MinPeakProminence', 0.06);
onset_samples = [1; locs; length(y)]; % 添加开始和结束点

%% 音高检测
num_notes = length(onset_samples)-1;
pitches = zeros(num_notes, 1);
durations = zeros(num_notes, 1);
note_names = cell(num_notes, 1);

% 创建结果矩阵
result = zeros(num_notes, 4); % [起始时间, 持续时间, 音名, 频率]

figure;

for i = 1:num_notes
    start_idx = onset_samples(i);
    end_idx = onset_samples(i+1);
    durations(i) = (end_idx - start_idx)/fs;
    result(i,1) = t(start_idx); % 起始时间
    result(i,2) = durations(i); % 持续时间

    if durations(i) >= t_min
        note_seg = y(start_idx:end_idx);

        % 傅里叶变换分析
        note_rep = repmat(note_seg, 100, 1); % 重复信号以提高频率分辨率
        N = length(note_rep);
```

```
% 加窗并计算 FFT
window = hann(N);
note_fft = abs(fft(note_rep .* window));
note_fft(1) = 0; % 去除直流分量
note_fft = fftshift(note_fft);
note_fft = note_fft(N/2+1:end); % 取正频率部分

% 频率轴
f = (0:N/2-1)*fs/N;

% 绘制频谱图
subplot(ceil(num_notes/4), 4, i);
plot(f, note_fft);
xlabel('frequency/Hz');
ylabel('amp');
title(sprintf('音符 %d 频谱', i));
xlim([0, f_max]);
grid on;

% 寻找频谱峰值
[peak_amps, peak_loc] = findpeaks(note_fft, ...
    'MinPeakDistance', round(N*0.05/fs), ...
    'MinPeakHeight', max(note_fft)*0.02, ...
    'MinPeakProminence', 0.04);

if ~isempty(peak_loc)
    % 按幅度排序
    [~, sort_idx] = sort(peak_amps, 'descend');
    sorted_loc = peak_loc(sort_idx);
    sorted_freqs = f(sorted_loc);

    % 找出基频
    f0 = 0;
    for k = 1:length(sorted_freqs)
        temp = sorted_freqs(k);
        if temp < f_min || temp > f_max
            continue;
        end

        % 检查是否是其他频率的谐波
        is_harmonic = false;
        for m = 1:k-1
            ratio = temp / sorted_freqs(m);
```

```
        if abs(ratio - round(ratio)) < tolerance
            is_harmonic = true;
            break;
        end
    end

    if ~is_harmonic
        f0 = temp;
        break;
    end
end

% 检查频率是否在合理范围内
if f0 >= f_min && f0 <= f_max
    pitches(i) = f0;
    note_names{i} = freq_to_note(f0);
else
    note_names{i} = 'Rest';
end

else
    note_names{i} = 'Rest';
end

result(i,3) = pitches(i);
if pitches(i) > 0
    result(i,4) = str2double(note_names{i}(2:end)); % 音高数字
部分
end
else
    note_names{i} = 'Rest';
end
end

%% 结果输出
fprintf('%-7s %-8s %-8s %-10s\n', '起始时间', '持续时间/s', '音名', '
频率/Hz');
fprintf('-----\n');

for i = 1:num_notes
    if pitches(i) > 0
        fprintf('%-10.2f %-10.2f %-10s %-10.2f\n', ...
            result(i,1), result(i,2), note_names{i}, result(i,3));
    else
        fprintf('%-10.2f %-10.2f %-10s\n', ...
```

```
        result(i,1), result(i,2), 'Rest');
    end
end

%% 可视化
figure;
plot(t, y);
hold on;
for i = 1:length(onset_samples)
    xline(t(onset_samples(i)), 'r--', 'LineWidth', 1);
end
title('节拍划分');
xlabel('t/s');
ylabel('amplitude');

%% 保存结果到文件
save('ans.mat', 'result');

%% 创建包含所有信息的结果表格
final_result = table();
final_result.StartTime = result(:,1);
final_result.Duration = result(:,2);
final_result.Frequency = result(:,3);
final_result.NoteName = note_names;

writetable(final_result, 'ans.csv');
end

function note_name = freq_to_note(freq)
    % 十二平均律音高转换
    note_names = {'C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B'};
    if freq <= 0
        note_name = 'Rest';
        return;
    end

    semitone = round(12 * log2(freq / 440) + 69);
    octave = floor(semitone / 12) - 1;
    note_idx = mod(semitone-1, 12) + 1;
    note_name = [note_names{note_idx} num2str(octave)];
end
```

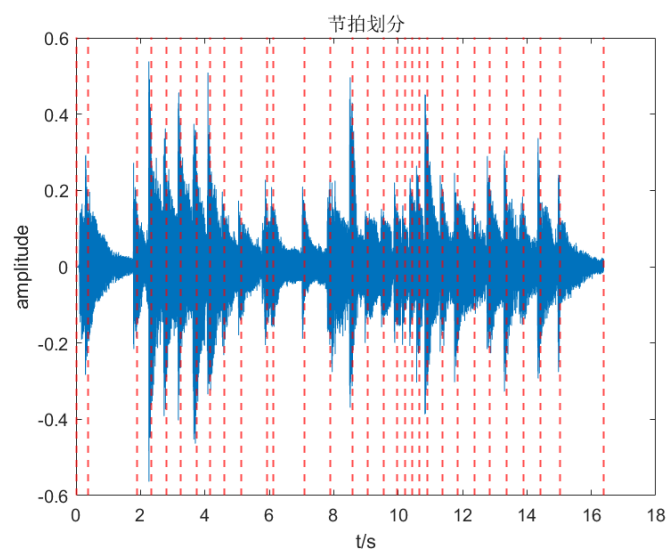

(3) 运行结果

得到音符频率、音名、持续时间如下，一共识别出 30 个音：

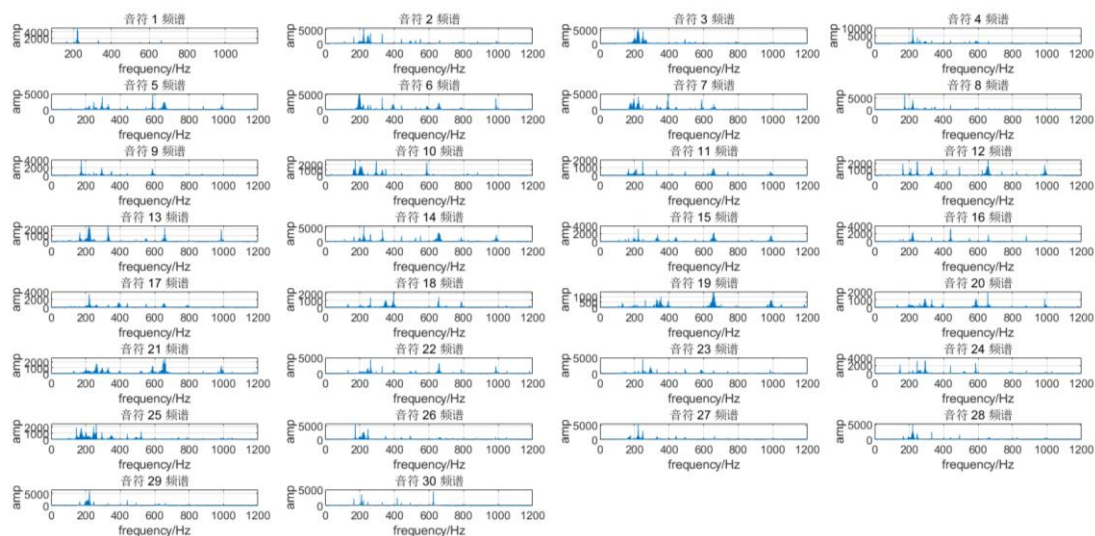
```
>> HW1_9
```

起始时间	持续时间/s	音名	频率/Hz
0.00	0.37	G#3	219.36
0.37	1.53	G#3	221.73
1.90	0.43	G#3	220.94
2.34	0.47	G#3	221.33
2.80	0.44	C#5	588.83
3.25	0.49	F#3	197.30
3.74	0.43	F#4	392.84
4.17	0.44	E3	174.06
4.61	0.52	E3	175.24
5.14	0.80	E3	174.60
5.93	0.20	A#3	246.85
6.13	0.97	D#5	658.17
7.10	0.80	G#3	220.53
7.90	0.67	G#3	222.26
8.57	0.48	G#3	221.41
9.05	0.51	G#4	441.72
9.56	0.39	G#3	220.25
9.96	0.25	F#4	394.76
10.21	0.23	D#5	658.77
10.44	0.22	D#5	656.72
10.65	0.26	D#5	659.83
10.91	0.48	B4	262.71
11.39	0.46	A#3	247.78
11.85	0.52	C#4	292.89
12.38	0.46	B4	261.20
12.84	0.53	E3	174.44
13.37	0.52	G#3	220.77
13.89	0.53	G#3	220.96
14.42	0.62	G#3	222.04
15.04	1.34	D5	624.88

划分的节拍如下图所示，分割线大多位于包络峰值，肉眼发现分割结果符合实际：



30 个音符的傅里叶变换频谱图如下：



四、 基于傅里叶级数的合成音乐

1. 用计算出的傅里叶级数重新演奏带谐波的《东方红》

(1) 设计思路

仍采用谐波叠加的方式合成音乐，利用傅里叶方法计算出的吉他谐波数据 `freq_harm` 和 `amp`，将目标音符的基频与吉他基频的比例关系 `r` 应用于所有谐波，实现频率缩放，保留原始谐波结构，以保持原始吉他音色。将生成的音乐保存在 `1_10.wav` 中。

(2) 核心代码

```
fs = 8000;
beattime = 0.5;

freq = [523.25, 523.25, 587.33, 392, 349.23, 349.23, 293.66, 392];
beatnum = [1, 0.5, 0.5, 2, 1, 0.5, 0.5, 2];
rhythm = [];

% 从 Guitar.MAT 分析得到的频率和振幅
freq_harm = [329.1, 658.4, 987.5, 1316.9, 1646.2, 1975.3, 2304.4,
2633.8, 2962.9, 3292.0];
amp = [0.0439076, 0.0791423, 0.0438014, 0.0594127, 0.00198038,
0.00587942, 0.0175337, 0.00655073, 0.00677967, 0.00188444];
amp = amp / max(amp); % 归一化
```

```
for i = 1:length(beatnum)
    t = beatnum(i) * beattime;
    t = linspace(0, t, t * fs);
    env = envel(t,3);

    % 初始化当前音符信号
    temp = zeros(size(t));
    r = freq(i) / freq_harm(1); % 计算基频与吉他基频的比例

    % 叠加所有谐波成分
    for i_harm = 1:length(freq_harm)
        freq_new = freq_harm(i_harm) * r; % 将吉他谐波频率按比例调整到目标
        % 音符频率
        temp = temp + amp(i_harm) * sin(2 * pi * freq_new * t); % 添加谐
        % 波成分
    end

    temp = temp .* env;
    rhythm = [rhythm, temp];
end

rhythm = rhythm / max(abs(rhythm));

sound(rhythm, fs);
audiowrite('1_10.wav', rhythm, fs);
```

(3) 运行结果

相较未引入吉他音色傅里叶变换的结果，音乐中有明显弦乐元素，但吉他声并不明显。

2. 根据获得的音调频率信息重新演奏带谐波的《东方红》

(1) 设计思路

首先，调用之前节拍划分和音调分析的代码，实现自动分析吉他乐器的音色频率特征，并保存为特征库 `guitar_features.mat`。

合成音乐时，根据目标音符的频率，匹配最接近的谐波比例。在旋律中叠加多个谐波，并保留吉他特有的频率特性。此外，使用更适合吉他音色的包络，引入拨弦噪声函数，以增强吉他音色的真实感。将合成出的音乐保存在 `1_11.wav` 中。

(2) 核心代码

```
% 吉他音色合成
for i = 1:length(note)
    target_note = note{i};
    t = beatnum(i) * beattime;
    t = linspace(0, t, round(t*fs));

    target_freq = note2freq(target_note); % 获取目标频率
    [~, idx] = min(abs(analyzed_freqs - target_freq)); % 选择最接近的
    吉他音色特征

    if analyzed_freqs(idx) > 0
        % 使用分析得到的谐波特征
        harm_ratios = harm_data{idx}(2:end)/harm_data{idx}(1); % 相对
        基频的比率
        harm_amps = [1.0, 0.6, 0.3, 0.15, 0.07]; % 谐波幅度
    else
        % 默认值
        harm_ratios = [2.05, 3.1, 4.2];
        harm_amps = [0.6, 0.3, 0.1];
    end

    % 合成吉他音色
    note_signal = sin(2*pi*target_freq*t); % 基频

    % 添加谐波成分
    for h = 1:min(3, length(harm_ratios)) % 最多使用 3 个谐波
        freq_var = 0.005 * randn(); % 轻微随机失谐
        note_signal = note_signal + ...
            harm_amps(h) *
            sin(2*pi*target_freq*harm_ratios(h)*(1+freq_var)*t);
    end

    % 应用吉他包络
    env = envel(t);
    note_signal = note_signal .* env;

    % 添加拨弦噪声
    noise_env = min(1, 3*env); % 噪声包络更短促
    note_signal = note_signal + 0.03*noise_env.*randn(size(t));

    rhythm = [rhythm, note_signal];
end
```

```
% 后期处理
rhythm = rhythm / max(abs(rhythm));
rhythm = effect(rhythm, fs);

% 播放和保存
sound(rhythm, fs);
audiowrite('1_11.wav', rhythm, fs);
```

(3) 运行结果

相较上一问，吉他音色更明显了一些，但仍与真实吉他音色存在一定的偏差。推测原因可能是这一句音乐中使用到的谐波分量并没有完全被特征库所覆盖，使用临近的信息代替可能导致了合成存在偏差。

3. 用图形界面分装前述功能

(1) 设计思路

为实现一个基于 MATLAB 图形界面的音乐合成器系统，需要合理利用已经撰写好的代码，对各个模块的功能进行封装。合成音乐的核心原理为：先通过分析真实乐器音频样本提取音色特征，再根据用户输入的音名、节拍、速度和采样率参数合成新的音乐。

因此，设计的应用系统首先提供文件选择界面加载乐器音频（如吉他），分析模块提取音频中的基频和谐波特征，构建对应乐器的音色模型。用户可在界面输入音符序列和对应节拍，系统将自动计算目标频率并匹配最接近的预存音色特征，通过叠加谐波生成每个音符信号，并应用包络增加音色的真实性。最终，通过 `sound` 函数播放合成的音乐，用户也可以选择保存为 `wav` 文件。

整个系统通过 GUI 组件实现交互式操作，将音频分析、乐谱解析和音乐合成等功能集成在统一界面中。

(2) 核心代码

```
function HW1_12()
% 创建主窗口
fig = figure('Name', '音乐合成器', 'Position', [100 100 800 600],
'NumberTitle', 'off');

% 输入文件选择
uicontrol('Style', 'text', 'Position', [20 550 150 20], 'String', '
选择乐器音频文件:');
```

```
file_path = uicontrol('Style', 'edit', 'Position', [180 550 400 20],  
'String', 'fmt.wav');  
uicontrol('Style', 'pushbutton', 'Position', [590 550 80 20],  
'String', '浏览...',...  
    'Callback', @browse_file);  
  
% 乐谱输入区  
uicontrol('Style', 'text', 'Position', [20 480 150 20], 'String', '  
输入音符序列:');  
notes_input = uicontrol('Style', 'edit', 'Position', [180 480 400  
60],...  
    'String', 'C5 C5 D5 G4 | F4 F4 D4 G4', 'Max',  
3);  
  
uicontrol('Style', 'text', 'Position', [20 420 150 20], 'String', '  
输入节拍序列:');  
beats_input = uicontrol('Style', 'edit', 'Position', [180 420 400  
20],...  
    'String', '1 0.5 0.5 2 | 1 0.5 0.5 2');  
  
% 参数设置  
uicontrol('Style', 'text', 'Position', [20 380 150 20], 'String', '  
速度(BPM):');  
tempo_input = uicontrol('Style', 'edit', 'Position', [180 380 100  
20], 'String', '100');  
  
uicontrol('Style', 'text', 'Position', [300 380 100 20], 'String', '  
采样率(Hz):');  
fs_input = uicontrol('Style', 'edit', 'Position', [400 380 100 20],  
'String', '8000');  
  
% 控制按钮  
uicontrol('Style', 'pushbutton', 'Position', [180 320 120 30],  
'String', '分析乐器音色',...  
    'Callback', @analyze_instrument);  
uicontrol('Style', 'pushbutton', 'Position', [320 320 120 30],  
'String', '试听合成结果',...  
    'Callback', @synthesize_music);  
uicontrol('Style', 'pushbutton', 'Position', [460 320 120 30],  
'String', '保存为 WAV 文件',...  
    'Callback', @save_audio);  
  
% 结果显示区  
panel = uicontrol('Style', 'text', 'Position', [20 100 760 200],...
```

```
        'String', '准备就绪...', 'HorizontalAlignment',  
'left');  
  
% 音频分析结果存储  
feature = struct();  
rhythm = [];  
  
%% 回调函数  
function browse_file(~,~)  
    [file, path] = uigetfile('*.wav', '选择乐器音频文件');  
    if file ~= 0  
        set(file_path, 'String', fullfile(path, file));  
    end  
end  
  
%% 音色分析函数  
function analyze_instrument(~,~)  
    audio_file = get(file_path, 'String');  
    try  
        [y, fs] = audioread(audio_file);  
        set(panel, 'String', '正在分析乐器音色特征...');  
        drawnow;  
  
        result = HW1_9(); % 调用任务(9)的分析音色代码  
  
        % 提取特征  
        feature.freqs = result(:,3);  
        feature.durations = result(:,2);  
  
        % 计算谐波特征  
        harm_data = cell(length(feature.freqs),1);  
        for i = 1:length(feature.freqs)  
            if feature.freqs(i) > 0  
                harm_data{i} = feature.freqs(i) * [1, 2.1, 3.2];  
            end  
        end  
        feature.harmonics = harm_data;  
  
        set(panel, 'String',...  
            sprintf('分析完成! \n 共提取%d 个有效音符特征\n 平均频率: %.2f  
Hz',...  
                sum(feature.freqs>0),...  
                mean(feature.freqs(feature.freqs>0))));  
    catch e
```

```
        set(panel, 'String', ['分析失败: ' e.message]);
    end
end

%% 音乐合成函数
function synthesize_music(~,~)
    if isempty(fieldnames(feature))
        set(panel, 'String', '请先分析乐器音色! ');
        return;
    end

    try
        % 解析输入
        notes = strsplit(strrep(get(notes_input, 'String'), '|',
'')); % 移除小节线
        beats = strsplit(strrep(get(beats_input, 'String'), '|',
''));

        tempo = str2double(get(tempo_input, 'String'));
        fs = str2double(get(fs_input, 'String'));

        % 验证输入
        if length(notes) ~= length(beats)
            set(panel, 'String', '错误: 音符和节拍数量不匹配! ');
            return;
        end

        set(panel, 'String', '正在合成音乐...');
        drawnow;

        % 合成参数
        beat_duration = 60/tempo;
        rhythm = [];

        % 合成每个音符
        for i = 1:length(notes)
            note = strtrim(notes{i});
            t = str2double(beats{i}) * beat_duration;

            target_freq = note2freq(note); % 获取目标频率
            [~, idx] = min(abs(feature.freqs - target_freq)); % 查找
最接近的音色特征
            t = linspace(0, t, round(t*fs)); % 合成音符

            if feature.freqs(idx) > 0
```



```
        % 使用分析得到的谐波特征
        harm_ratios = feature.harmonics{idx} /
feature.harmonics{idx}(1);
        harm_amps = [1.0, 0.6, 0.3];
    else
        % 默认值
        harm_ratios = [1, 2, 3];
        harm_amps = [1.0, 0.5, 0.3];
    end

    % 合成音色
    note_signal = zeros(size(t));
    for h = 1:length(harm_ratios)
        freq = target_freq * harm_ratios(h);
        note_signal = note_signal + harm_amps(h) *
sin(2*pi*freq*t);
    end

    env = envel(t); % 使用 ADSR 包络
    note_signal = note_signal .* env;
    rhythm = [rhythm, note_signal];
end

% 后期处理
rhythm = rhythm / max(abs(rhythm));

% 播放音频
sound(rhythm, fs);
set(panel, 'String', '合成完成! 点击"保存为 WAV 文件"可保存音频
');
catch e
    set(panel, 'String', ['合成失败: ' e.message]);
end
end

function save_audio(~,~)
    if isempty(rhythm)
        set(panel, 'String', '没有可保存的音频数据! ');
        return;
    end

    [file, path] = uiputfile('*.wav', '保存合成音频');
    if file ~= 0
        fs = str2double(get(fs_input, 'String'));
    end
end
```

```
        audiowrite(fullfile(path, file), rhythm, fs);
        set(panel, 'String', ['音频已保存至: ' fullfile(path, file)]);
    end
end

% 辅助函数
function freq = note2freq(note)
    note_names =
{'C','C#','D','D#','E','F','F#','G','G#','A','A#','B'};
    if contains(note, '#')
        note_str = note(1:2);
        octave = str2double(note(3:end));
    else
        note_str = note(1);
        octave = str2double(note(2:end));
    end
    note_num = find(strcmp(note_names, note_str));
    freq = 440 * 2^((note_num + (octave-4)*12 - 9)/12);
end

function env = envel(t)
    len = length(t);
    env = zeros(size(t));
    attack_end = round(0.2*len);
    decay_end = round(0.3*len);
    sustain_end = round(0.65*len);

    if attack_end > 0 % Attack
        env(1:attack_end) = linspace(0,1,attack_end);
    end
    if decay_end > attack_end % Decay
        env(attack_end+1:decay_end) = linspace(1,0.3,decay_end-
attack_end);
    end
    if sustain_end > decay_end % Sustain
        env(decay_end+1:sustain_end) = 0.3;
    end
    if len > sustain_end % Release
        env(sustain_end+1:end) = linspace(0.3,0,len-sustain_end);
    end
end
end
```

（3）运行结果

封装后的使用界面如图所示，有较充分的提示词引导。默认填充的值为本次 MATLAB 作业中提供的吉他音色与《东方红》的合成要求。

经过试验，分析器乐音色、合成对应音乐并保存为 wav 文件的功能均能实现，且用户可以自由更改输入的音频文件、音符序列和节拍序列（符合输入格式即可）、音乐的播放速度以及采样率。



The image shows a MATLAB GUI for music synthesis. It has a light gray background. At the top, there's a label '选择乐器音频文件:' followed by a text box containing 'fmt.wav' and a '浏览...' button. Below that, there's a label '输入音符序列:' followed by a text box containing 'C5 C5 D5 G4 | F4 F4 D4 G4'. Then, a label '输入节拍序列:' followed by a text box containing '1 0.5 0.5 2 | 1 0.5 0.5 2'. Below these, there are two labels: '速度(BPM):' with a text box containing '100', and '采样率(Hz):' with a text box containing '8000'. At the bottom, there are three buttons: '分析乐器音色', '试听合成结果', and '保存为WAV文件'. In the bottom left corner, there's a label '准备就绪...'.

五、 实验总结

本次 MATLAB 大作业通过对基本乐理知识的介绍和引入，一步步指导我结合信号与系统的知识撰写 MATLAB 代码，实现了音色分析和音乐合成的全过程。首先，通过采样和定义频率与持续时间，就可以发出声音；为了使合成的音乐更贴合实际，可以引入包络；而为了丰富音乐的音色，可以通过分析真实乐器演奏的声音，利用傅里叶变换等方法将其变为频率特征，进而合成出某个乐器的声音。

通过本次实验，我对傅里叶变换和频率有了更深入的理解，也对合成电子音乐形成了基本认知，并提升了我撰写 MATLAB 代码的能力，收获颇丰。